

DOCUMENTATION
CONTEXTS DOCUMENTATION

Presentado por:

Isabella Garces Acosta
Diego Steven Pinzon Yossa
Juan Camilo Rosero Satisfesteban
Sharick Yelixa Torres Monroy
Laura Sofia Vargas Rodriguez

Profesora:

Liliana Marcela Olarte Mesa



Universidad Nacional de Colombia
Facultad de ingeniería
Departamento de Ingeniería de sistemas e industrial
2025



CONTENT

1. Notification context (React Context Provider)
2. Itinerary context (Primary Business Logic Context)
3. Users context (Authentication & Profile management)
4. Mock itinerary context (Development & testing provider)



Notification context

This Notification Context (NotificationContext.tsx) provides a centralized notification system for the application using react-hot-toast for UI notifications and local storage for persisting user preferences. It offers four notification types (success, error, info, warning) with custom styling and icons from Lucide React. The context manages user-configurable notification settings (email notifications, trip updates, recommendations) that are saved to localStorage. A specialized showItinerarySaved method creates rich, interactive notifications with action buttons when itineraries are successfully saved. The implementation includes conditional notification display based on user settings, ensuring users only receive notifications they've opted into. This context serves as a comprehensive notification hub that coordinates user feedback throughout the application with consistent styling and behavior.

Itinerary context

This comprehensive Itinerary Context (ItineraryContext.tsx) is the core business logic provider for the entire travel itinerary system. It manages the complete lifecycle of itinerary creation, modification, and persistence with extensive state management for current itineraries, user collections, and variant options. The context integrates with backend services for itinerary generation, provides sophisticated day and item manipulation (add, delete, move, replace), and handles real-time price recalculation. It features Firebase authentication integration for user-specific operations, automatic map marker generation from itinerary locations, and complex error handling with user feedback via the notification context. With 20+ methods covering everything from basic CRUD operations to advanced itinerary variant selection, this context orchestrates the entire travel planning experience while maintaining synchronization between UI state, backend APIs, and user authentication.

User context

This User Context (UserContext.tsx) manages user authentication state and profile data synchronization between Firebase Authentication and the application's backend user database. It establishes a real-time connection to Firebase auth state changes, automatically fetching corresponding user profile data from the backend API when authentication status changes. The context provides methods for refreshing user data, updating local state, and clearing user information on logout. It maintains a unified user object combining Firebase authentication data with application-specific profile information (onboarding status, origin city, profile image). This context serves as the single source of truth for user identity throughout the application, ensuring consistent access to both authentication state and user profile data across all components.



Mock itinerary context

This Mock Itinerary Context (MockItineraryContext.tsx) provides a fully-featured, hardcoded itinerary dataset for development, testing, and demonstration purposes. It creates a realistic 4-day Paris travel itinerary with complete details including flights, accommodations, restaurants, and tourist attractions. Each itinerary item includes rich structured data with coordinates, pricing, descriptions, and category-specific details. The context serves as a reliable data source during development when backend APIs may be unavailable, enabling frontend development and UI testing without dependency on external services. It demonstrates the complete data structure expected from the itinerary generation system and provides an excellent example for testing itinerary visualization, map integration, and day-by-day itinerary presentation components.